Java Backend Architecture

Interview Series

Chapter 11.5
Two-Phase Locking & Deadlocks

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

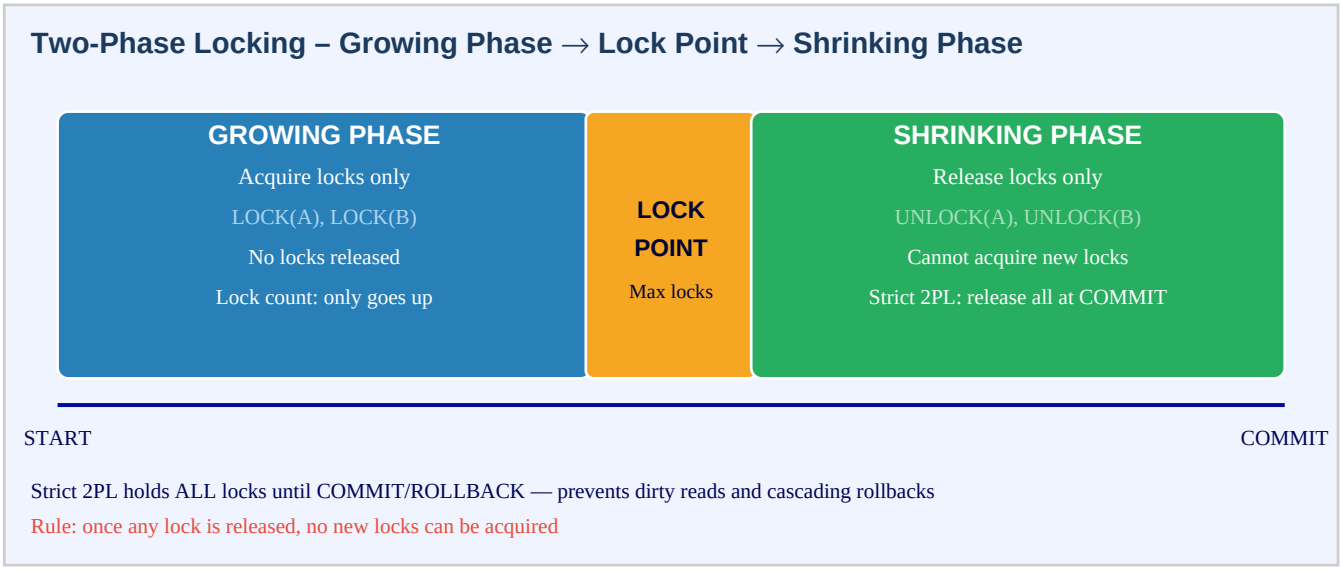
Chapter 11.5 – Two-Phase Locking & Deadlocks

Introduction

Two-Phase Locking (2PL) is the classical concurrency control protocol ensuring serializability in database transactions. It divides transaction execution into two strict phases: a growing phase where locks are acquired (but never released) and a shrinking phase where locks are released (but no new locks acquired). The point between the phases — the lock point — represents the maximum number of locks held. Strict 2PL, used by most commercial databases including PostgreSQL, holds all locks until commit or rollback, preventing cascading rollbacks.

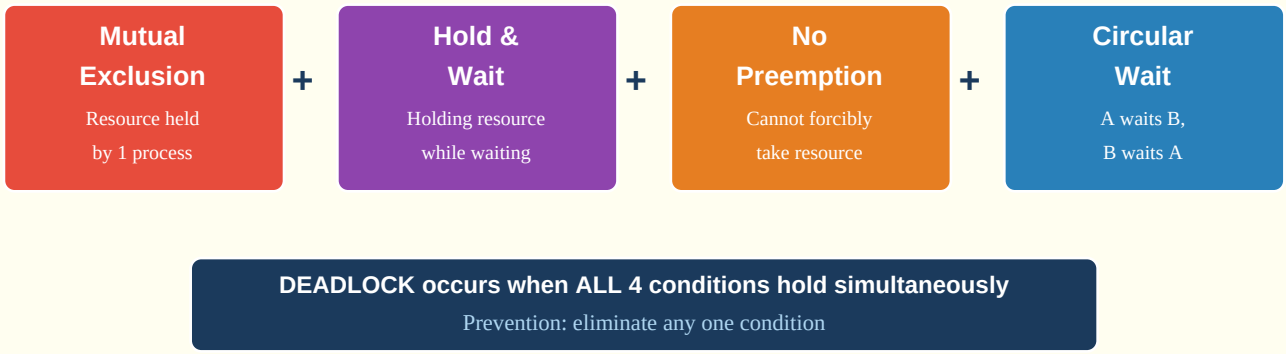
The main hazard of locking protocols is deadlock — a cycle of transactions each waiting for a lock held by another. Deadlocks require all four Coffman conditions simultaneously: mutual exclusion, hold-and-wait, no preemption, and circular wait. PostgreSQL detects deadlocks via a background wait-for graph analysis (triggered after deadlock_timeout, default 1 second) and resolves them by aborting one victim transaction with SQLSTATE 40P01. Java applications must detect and retry on this error code.

Two-Phase Locking: Growing & Shrinking Phases



Coffman Conditions for Deadlock

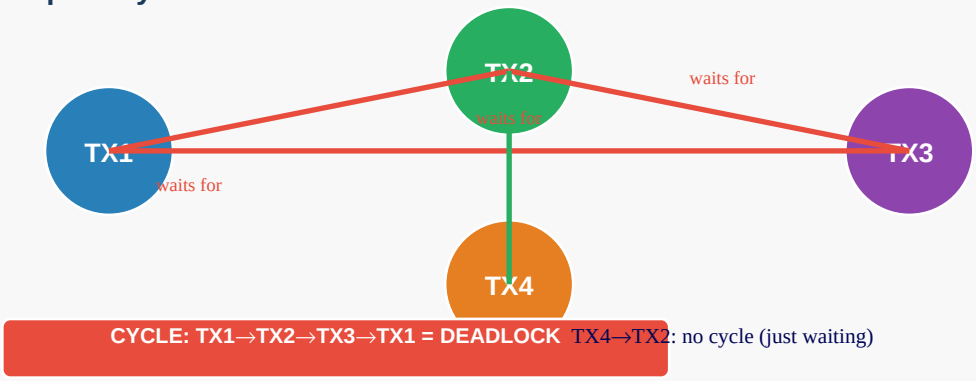
Coffman Conditions for Deadlock (all 4 must hold)



PostgreSQL: detects via wait-for graph, kills one victim (SQLSTATE 40P01)

Wait-For Graph: Cycle = Deadlock

Wait-For Graph – Cycle Detection = Deadlock



Key Definitions

2PL (Two-Phase Locking)	Protocol: acquire all needed locks (growing) then release (shrinking); ensures serializability
Growing phase	Transaction phase where only lock acquisitions are allowed; no releases
Shrinking phase	Transaction phase after first lock release; no new locks may be acquired
Lock point	Moment when the transaction holds its maximum set of locks; boundary between phases
Strict 2PL	All locks held until COMMIT or ROLLBACK; prevents cascading rollbacks; used by PostgreSQL
Deadlock	Cyclic wait where TX1 waits for TX2 and TX2 waits for TX1 (or longer cycle)
Coffman conditions	Four necessary conditions for deadlock: mutual exclusion, hold-and-wait, no preemption, circular wait

Wait-for graph	Directed graph where an edge TX_A→TX_B means A is waiting for a lock held by B; cycle = deadlock
Deadlock detection	Algorithm that periodically checks the wait-for graph for cycles; kills a victim
Deadlock prevention	Design strategies that eliminate one Coffman condition to make deadlocks impossible
deadlock_timeout	PostgreSQL GUC (default 1s); after this time waiting on a lock, deadlock detection runs
SQLSTATE 40P01	PostgreSQL error code for deadlock detected; application must catch and retry
Lock ordering	Prevention strategy: always acquire locks in same global order to eliminate circular wait
Timeout prevention	Prevention strategy: abort transactions waiting too long for a lock (statement_timeout/lock_timeout)
Victim selection	When a deadlock cycle is detected, one transaction is chosen as victim and aborted

Deadlock Detection vs Prevention vs Avoidance

Strategy	Mechanism	Overhead	Example
Detection	Wait-for graph cycle check	Post-hoc; victim aborted	PostgreSQL deadlock_timeout
Prevention (ordering)	Consistent lock order	Developer discipline	Always lock A before B
Prevention (timeout)	Abort after lock_timeout	Wasted work on timeout	SET lock_timeout=500ms
Avoidance (wound-wait)	Priority-based preemption	Complex; rare in OLTP	Older TX wounds younger
JDBC batch ordering	Sort entities before save	Minimal	Sort by PK before saveAll

Code Examples

1. PostgreSQL deadlock scenario and SQLSTATE 40P01

```
-- Session 1: -- Session 2: BEGIN; BEGIN; UPDATE accounts SET balance=balance-100 WHERE id=1;
-- locks row 1 UPDATE accounts SET balance=balance-50 WHERE id=2; -- locks row 2 UPDATE
accounts SET balance=balance+100 WHERE id=2; -- WAITS for row 2 UPDATE accounts SET
balance=balance+50 WHERE id=1; -- WAITS for row 1 -- After deadlock_timeout (1s): -- PostgreSQL
detects cycle: Session1→Session2→Session1 -- Aborts ONE session (the victim) with: -- ERROR:
deadlock detected -- DETAIL: Process 12345 waits for ShareLock on transaction 67890 -- HINT:
See server log for query details. -- SQLSTATE: 40P01 -- Java: catch and retry try {
jdbcTemplate.update("UPDATE accounts ..."); } catch (DeadlockLoserDataAccessException e) { //
Spring wraps 40P01 as DeadlockLoserDataAccessException retryWithBackoff(); }
```

2. Lock ordering prevention – always acquire in same order

```
// DEADLOCK-PRONE: different transaction order different lock order // TX1: lock(account1),
then lock(account2) // TX2: lock(account2), then lock(account1) → circular wait! //
DEADLOCK-FREE: always lock accounts in ascending ID order @Transactional public void
transfer(String fromId, String toId, BigDecimal amount) { // Determine consistent lock order by
ID String firstId = fromId.compareTo(toId) < 0 ? fromId : toId; String secondId =
```

```
fromId.compareTo(toId) < 0 ? toId : fromId; // Always acquire locks in the same order regardless of transfer direction Account first = accountRepo.findByIdForUpdate(firstId); // SELECT FOR UPDATE Account second = accountRepo.findByIdForUpdate(secondId); Account from = fromId.equals(firstId) ? first : second; Account to = fromId.equals(firstId) ? second : first; if (from.getBalance().compareTo(amount) < 0) { throw new InsufficientFundsException(); } from.debit(amount); to.credit(amount); accountRepo.saveAll(List.of(from, to)); } // Repository: // @Lock(LockModeType.PESSIMISTIC_WRITE) // @Query("SELECT a FROM Account a WHERE a.id = :id") // Account findByIdForUpdate(@Param("id") String id);
```

3. JDBC batch save ordering to prevent deadlocks

```
// When saving multiple entities in a batch, sort by primary key // to ensure all transactions acquire row locks in the same order @Service @Transactional public class OrderProcessingService { private final OrderItemRepository itemRepo; public void updateOrderItems(List<OrderItem> items) { // Sort by ID before batch save → consistent lock acquisition order List<OrderItem> sorted = items.stream() .sorted(Comparator.comparing(OrderItem::getId)) .collect(Collectors.toList()); itemRepo.saveAll(sorted); // Hibernate flushes in sorted order } } // Without sorting: // TX1 updates items [id=5, id=3] → locks row 5, then waits for row 3 // TX2 updates items [id=3, id=5] → locks row 3, then waits for row 5 → DEADLOCK // With sorting (both lock id=3 first, then id=5): // TX1 locks row 3 → TX2 waits → TX1 locks row 5, commits → TX2 proceeds
```

4. Spring retry on deadlock (40P01)

```
@Configuration @EnableRetry public class RetryConfig {} @Service public class TransactionService { @Retryable( retryFor = { DeadlockLoserDataAccessException.class, CannotAcquireLockException.class }, maxAttempts = 3, backoff = @Backoff(delay = 100, multiplier = 2, random = true) ) @Transactional public void performTransfer(TransferCommand cmd) { // If this transaction is chosen as deadlock victim → 40P01 // Spring wraps as DeadlockLoserDataAccessException // @Retryable catches it and retries up to 3 times with backoff accountService.debit(cmd.getFromAccountId(), cmd.getAmount()); accountService.credit(cmd.getToAccountId(), cmd.getAmount()); } @Recover public void recoverFromDeadlock(DeadlockLoserDataAccessException ex, TransferCommand cmd) { log.error("Transfer {} failed after retries due to deadlock", cmd.getId(), ex); throw new TransferFailedException("Could not complete transfer after retries"); } }
```

5. lock_timeout and statement_timeout for deadlock prevention

```
-- PostgreSQL: set lock timeout to prevent indefinite waiting -- If a lock cannot be acquired within the timeout, throw error (NOT deadlock, but prevention) -- Per-transaction setting: BEGIN; SET LOCAL lock_timeout = '500ms'; -- error if lock not acquired within 500ms SET LOCAL statement_timeout = '5s'; -- error if entire statement takes > 5s UPDATE products SET stock = stock - 1 WHERE id = ?; COMMIT; -- SQLSTATE: 55P03 (lock_not_available) on lock timeout -- SQLSTATE: 57014 (query_canceled) on statement timeout // Java: catch lock timeout @Transactional public void decrementStock(String productId) { try { jdbcTemplate.execute("SET LOCAL lock_timeout = '200ms'"); productRepo.decrementStock(productId); } catch (QueryTimeoutException | LockTimeoutException e) { // Lock not available → retry with backoff or return 503 throw new ResourceLockedException("Product inventory temporarily locked", e); } }
```

6. Strict 2PL vs Basic 2PL – cascading rollback example

```
-- Basic 2PL (allows releasing locks before commit) — DANGEROUS: -- TX1: LOCK(A), READ(A), UNLOCK(A), ... TX2 reads A here, TX1 later aborts -- TX2 read A when TX1 hadn't committed → dirty read → TX2 must also abort (cascade) -- Strict 2PL (PostgreSQL default): hold all locks until commit -- TX1: LOCK(A), READ(A), ... , COMMIT/ROLLBACK → UNLOCK(A) -- TX2 must wait for
```

```
TX1 to commit before reading A → no cascading rollbacks -- In practice, PostgreSQL uses Strict 2PL for UPDATE/DELETE locks: -- Row lock acquired on first read-for-update/write -- Released only when transaction ends (COMMIT or ROLLBACK) -- This is why long-running transactions (e.g., batch jobs) block other transactions: -- SELECT FOR UPDATE → lock held for entire transaction duration -- Solution: keep transactions short; do heavy computation outside of transaction
```

Interview Questions & Answers

Q1. Explain Two-Phase Locking and why it guarantees serializability.

Two-Phase Locking (2PL) divides each transaction into two phases: (1) Growing phase — the transaction may acquire locks but may NOT release any lock. (2) Shrinking phase — the transaction may release locks but may NOT acquire any new lock. This protocol guarantees serializability because: if TX_A holds a lock at the lock point (maximum locks held), any other transaction TX_B that conflicts with A must either wait until A releases its locks (in A's shrinking phase) or have obtained the lock before A entered its growing phase. This ordering creates a conflict-serializable schedule equivalent to some serial execution. The key invariant: once a lock is released, the transaction cannot 'go back' and acquire a conflicting lock held by another transaction.

Q2. What is Strict 2PL and how does it differ from basic 2PL?

Basic 2PL: locks can be released before commit as long as the two-phase rule is respected (no new locks after first release). Strict 2PL: ALL locks are held until the transaction commits or rolls back. Strict 2PL is stronger: it prevents cascading rollbacks. In basic 2PL, if TX_A releases a lock and TX_B reads the unlocked data, then TX_A aborts — TX_B has read uncommitted/aborted data and must also abort (cascade). In Strict 2PL, TX_B cannot read TX_A's data until TX_A commits, so TX_B's data is always from committed transactions. PostgreSQL implements Strict 2PL: row locks are held until commit/rollback.

Q3. What are the four Coffman conditions for deadlock?

All four must hold simultaneously for deadlock to occur: (1) Mutual Exclusion — a resource can be held by only one transaction at a time (row lock is exclusive). (2) Hold and Wait — a transaction holds at least one lock and is waiting to acquire another lock held by a different transaction. (3) No Preemption — a transaction's locks cannot be forcibly taken away; the holder must release voluntarily. (4) Circular Wait — there exists a cycle in the wait-for graph: TX_1 waits for TX_2, TX_2 waits for TX_3, ..., TX_n waits for TX_1. Deadlock prevention works by designing the system to violate at least one of these four conditions.

Q4. How does PostgreSQL detect deadlocks?

PostgreSQL detects deadlocks using a wait-for graph: directed edges represent 'transaction A is waiting for a lock held by transaction B'. When a transaction waits for a lock (its LOCK call blocks), PostgreSQL starts a timer equal to deadlock_timeout (default 1 second). After deadlock_timeout elapses, the lock manager checks for cycles in the wait-for graph. If a cycle is found, PostgreSQL selects a victim (typically the transaction that would be cheapest to abort — often the one that started most recently) and cancels it with SQLSTATE 40P01: 'deadlock detected'. The victim's transaction is rolled back and it receives an error. Other transactions in the cycle can then proceed.

Q5. What is SQLSTATE 40P01 and how should Java code handle it?

SQLSTATE 40P01 is PostgreSQL's error code for 'deadlock detected'. When PostgreSQL aborts a victim transaction due to a deadlock, the victim receives this error. Spring JDBC translates 40P01 to DeadlockLoserDataAccessException (a subclass of ConcurrencyFailureException). Handling strategy: (1) Catch DeadlockLoserDataAccessException. (2) Optionally log it at DEBUG level (deadlocks are expected under load). (3) Clear any caches or EntityManager state. (4) Retry the transaction after a short randomized backoff (50-200ms). (5) Cap retries at 3-5 attempts. (6) After max retries, propagate as a business error

(HTTP 409 or 503). Use @Retryable with Spring Retry for clean declarative retry.

Q6. How does lock ordering prevent deadlocks?

Lock ordering eliminates the circular wait Coffman condition. If every transaction acquires locks on resources in the same globally consistent order (e.g., ascending row ID), circular waits cannot form. Example: TX_1 must lock rows {id=3, id=5} and TX_2 must lock rows {id=5, id=3}. Without ordering: TX_1 locks 3, TX_2 locks 5, TX_1 waits for 5, TX_2 waits for 3 → deadlock. With ordering (always lock smaller ID first): both TX_1 and TX_2 try to lock 3 first → one succeeds, the other waits → no cycle forms. Implementation: sort entity IDs before saveAll(), sort account IDs before fund transfers. This is a preventive strategy requiring code discipline — no runtime overhead.

Q7. Why does JDBC batch order matter for deadlock prevention?

When Hibernate/Spring Data flushes a batch of entity updates, the SQL statements are issued in the order of the Java list. If two concurrent transactions update overlapping sets of rows in different orders, they can deadlock. Example: TX_1 calls saveAll([item_5, item_3]) — Hibernate issues UPDATE item_5, UPDATE item_3 (locks in that order). TX_2 calls saveAll([item_3, item_5]) — issues UPDATE item_3, UPDATE item_5. TX_1 holds lock on item_5, TX_2 holds lock on item_3 → circular wait. Fix: sort the list by ID before saveAll. Hibernate Pro Tip: also ensure cascade order is consistent for parent-child relationships; inconsistent cascade insert order can also deadlock.

Q8. What is a wait-for graph and how is it used for deadlock detection?

A wait-for graph is a directed graph where each vertex is a transaction and an edge TX_A → TX_B means 'TX_A is waiting for a lock currently held by TX_B'. The graph is maintained by the lock manager and updated whenever a lock request blocks. Deadlock detection: scan the graph for cycles (using DFS or BFS). A cycle of length 2 (TX_A → TX_B → TX_A) is a two-transaction deadlock; longer cycles are multi-transaction deadlocks. PostgreSQL runs this detection after deadlock_timeout. The detection is not run continuously (too expensive) but triggered by lock waits exceeding the timeout. Once a cycle is found, one transaction is chosen as the victim and aborted.

Q9. How does deadlock timeout differ from lock timeout?

deadlock_timeout (PostgreSQL GUC, default 1s): the time a transaction waits for a lock before PostgreSQL checks for a deadlock cycle in the wait-for graph. If a cycle is found, one transaction is aborted with SQLSTATE 40P01. If no cycle, the transaction continues waiting. lock_timeout (connection/session parameter): the maximum time a statement waits for any lock. If the lock is not acquired within lock_timeout, the statement is immediately aborted with SQLSTATE 55P03 (lock_not_available) — regardless of deadlock. statement_timeout is even broader — it aborts any statement exceeding the timeout. lock_timeout is useful for preventing indefinite waits even when there is no deadlock.

Q10. Describe a real-world deadlock scenario in a Java banking application.

Classic fund transfer deadlock: Service A processes a transfer from Account 100 to Account 200. Service B simultaneously processes a transfer from Account 200 to Account 100. Both services use @Transactional and issue SELECT FOR UPDATE in their account order. Service A locks Account 100, Service B locks Account 200. Service A requests a lock on Account 200 (held by B) — blocks. Service B requests a lock on Account 100 (held by A) — blocks. Deadlock. After 1 second (deadlock_timeout), PostgreSQL detects the cycle, aborts one transaction (SQLSTATE 40P01), the other completes. Fix: sort account IDs before locking, so both services always lock the lower ID first, eliminating circular wait.

Q11. What is phantom read and is it related to 2PL?

A phantom read occurs when a transaction re-executes a range query and finds new rows that were inserted by another committed transaction between the two executions. Standard 2PL with row locks prevents lost

updates and dirty reads but does NOT prevent phantom reads — because a new row doesn't have an existing row lock to prevent its insertion. To prevent phantoms, databases use predicate locks or gap locks (MySQL InnoDB) that lock ranges rather than individual rows. Strict 2PL with predicate locking is called Strict Serializable and prevents all anomalies. PostgreSQL at SERIALIZABLE level uses SSI (Serializable Snapshot Isolation) to prevent phantoms without actual predicate locks.

Q12. How do you tune deadlock_timeout in PostgreSQL?

deadlock_timeout controls how long a transaction waits before the DB checks for deadlocks. Default: 1 second. If your application frequently deadlocks: (1) Set lower deadlock_timeout (e.g., 100ms) to detect and resolve deadlocks faster, reducing latency for victim transactions. (2) Set higher deadlock_timeout if deadlocks are rare and you prefer avoiding the overhead of frequent wait-for graph scans. Configuration: postgresql.conf (global) or SET deadlock_timeout = '100ms' per session. Monitor: pg_stat_activity.wait_event = 'Lock' shows currently blocked queries. pg_locks shows all active locks. log_lock_waits = on logs lock waits exceeding deadlock_timeout (useful for diagnosing deadlock patterns).

Q13. How does the wound-wait deadlock prevention algorithm work?

Wound-wait is a priority-based deadlock prevention algorithm based on transaction age. Each transaction is assigned a timestamp when it starts (older = higher priority). When TX_A requests a lock held by TX_B: if TX_A is OLDER than TX_B — TX_A 'wounds' TX_B (aborts TX_B, letting TX_A proceed). If TX_A is YOUNGER than TX_B — TX_A waits. This ensures no circular waits: a younger transaction never makes an older transaction wait. The tradeoff: younger transactions get aborted often (when older ones need their locks), causing wasted work. Wound-wait is rarely used in OLTP databases — PostgreSQL uses detection (wait-for graph) rather than prevention algorithms.

Q14. What is the difference between a shared lock and an exclusive lock in 2PL?

Shared lock (read lock, S-lock): multiple transactions can hold a shared lock on the same resource simultaneously. A transaction acquiring a shared lock can read the resource but not modify it. Exclusive lock (write lock, X-lock): only one transaction can hold an exclusive lock. No other transaction can hold any lock (shared or exclusive) on the resource while an exclusive lock is held. 2PL with shared/exclusive locks: reads acquire shared locks, writes acquire exclusive locks. The lock upgrade problem: a transaction holding a shared lock trying to upgrade to exclusive may deadlock with another transaction also trying to upgrade. Solution: acquire exclusive lock upfront if a write is anticipated.

Q15. How do you monitor for deadlocks and lock contention in production PostgreSQL?

Monitoring tools: (1) pg_stat_activity: shows current queries, wait_event_type='Lock', wait_event='relation'/'tuple'/'transactionid'. (2) pg_locks: shows all current lock requests and granted status. Join pg_locks with pg_stat_activity to see which queries are blocking/blocked. (3) log_lock_waits = on in postgresql.conf: logs any lock wait exceeding deadlock_timeout to the server log. (4) EXPLAIN ANALYZE: look for 'LockRows' and 'Seq Scan' on large tables (full table scan before FOR UPDATE). (5) Application metrics: track DeadlockLoserDataAccessException rate in Prometheus/Grafana. (6) Deadlock graph in logs: when log_min_messages = LOG, PostgreSQL logs the deadlock detail showing the involved transactions and locked objects.

Q16. What are the symptoms of 2PL causing performance problems in production?

Symptoms: (1) High wait_event='Lock' in pg_stat_activity — many queries waiting for locks. (2) Slow query response times correlating with high concurrency. (3) Frequent DeadlockLoserDataAccessException in application logs. (4) Connection pool exhaustion — threads holding connections while waiting for locks, exhausting the pool for other requests. (5) Lock queue depth growing: one slow transaction holds a lock, creating a queue of waiting transactions. Root causes: (1) Long-running transactions holding row locks. (2) N+1 SELECT FOR UPDATE in loops. (3) Full table scans with FOR UPDATE. Fixes: shorten transactions,

use lock_timeout, paginate bulk updates, use application-level partitioning to reduce lock overlap.

Q17. Explain how Hibernate session can cause unexpected deadlocks.

Hibernate's automatic dirty detection can issue UPDATEs in an unpredictable order at flush time, especially when entities are added to the session from different code paths. This causes lock acquisition order to vary between transactions, leading to circular waits. Also: Hibernate's second-level cache may evict/update cache entries in a different order than the DB writes, creating a window for inconsistency. Solutions: (1) Explicitly call session.flush() after each critical UPDATE to control timing. (2) Use sorted collections or sort entities before saveAll(). (3) Use @OrderBy on collections to ensure consistent update order. (4) Set hibernate.order_updates=true (Hibernate 5+) to sort SQL UPDATEs by entity type and ID before flushing — effectively automatic lock-ordering prevention.

Q18. What is the deadlock probability formula and how does it guide design?

Deadlock probability increases with: (1) Number of concurrent transactions (N) — more transactions = more potential cycles. (2) Number of locks per transaction (L) — more locks = higher chance of overlapping lock sets. (3) Transaction duration (T) — longer transactions hold locks longer, increasing overlap probability. (4) Degree of lock overlap (O) — transactions that frequently access the same rows. Rule of thumb: $P(\text{deadlock}) \propto N^2 \times L^2 \times T \times O$. To reduce deadlock probability: shorten transactions (reduce T), reduce lock scope (reduce L), partition data to reduce overlap (reduce O), and use optimistic locking for read-heavy paths (reduce N competing for exclusive locks).

Q19. How do distributed 2PL and local 2PL differ in a microservices architecture?

Local 2PL: each service uses 2PL within its own database. Works within a single bounded context. Distributed 2PL: extends 2PL across multiple database nodes/services. Requires a distributed lock manager (DLM) or two-phase commit (2PC) coordinator. Problems: (1) Network failures can leave the DLM in an uncertain state. (2) A coordinator crash blocks all participants (in-doubt transactions). (3) Deadlock detection requires a global wait-for graph spanning all nodes. In practice, distributed 2PL is rarely implemented in modern microservices. Instead: use Saga pattern (local transactions + compensation), or Outbox pattern (atomic write + event publishing), avoiding the need for distributed locking entirely.

Q20. How does hibernate.order_updates help prevent deadlocks?

hibernate.order_updates=true (set in Spring Boot: spring.jpa.properties.hibernate.order_updates=true) causes Hibernate to sort SQL UPDATE statements by entity type and primary key before flushing to the database. This ensures that all transactions acquire row locks in the same ascending ID order, eliminating circular wait conditions for same-entity updates. Similarly, hibernate.order_inserts=true sorts INSERTs by entity type and ID, enabling JDBC batch inserts (reduces round trips) and consistent lock order. Combined with spring.jpa.properties.hibernate.jdbc.batch_size=50, these settings both improve throughput and reduce deadlock probability. This is the easiest automated lock-ordering solution for Hibernate-based applications.